



**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

Oskar Tyniec 164028 2EF-DI P1

Realiacja sieci neuronowej uczonej algorytmem wstecznej propoagacji błędu, klasyfikującej choroby nerek. Badanie wpływu parametrów sieci na proces uczenia.

**Praca projektowa z przedmiotu Sztuczna
Inteligencja**

Rzeszów, 2021

Spis treści

1. Opis projektu	5
1.1. Założenia projektowe	5
1.2. Zestaw danych	5
1.3. Przygotowanie danych	6
2. Wstęp teoretyczny	8
2.1. Model sztucznego neuronu	8
2.2. Funkcja aktywacji	9
2.3. Model sieci wielowarstwowej	9
2.4. Algorytm wstecznej propagacji błędów	11
3. Realizacja sieci neuronowej	13
3.1. Opis skryptu	13
3.2. Skrypt dla zależności poprawności klasyfikacji od liczby neuronów w warstwach	13
3.3. Skrypt dla zależności poprawności klasyfikacji od wartości współczynnika uczenia	16
4. Eksperymenty	18
4.1. Eksperyment 1	18
4.2. Eksperyment 2	22
4.3. Eksperyment 3	25
4.4. Eksperyment 4	28
4.5. Eksperyment 5	30
4.6. Eksperyment 6	32
4.7. Eksperyment 7	34
4.8. Eksperyment 8	36
4.9. Eksperyment 9	38
4.10. Eksperyment 10	40
5. Wnioski	42
Literatura	43

1. Opis projektu

1.1. Założenia projektowe

Celem projektu jest realizacja sieci neuronowej uczącej się klasyfikowania chorób dla podanych objawów. Dodatkowym celem jest zbadanie wpływu poszczególnych parametrów sieci na proces uczenia się tej sieci. Sieć zrealizowana została przy użyciu programu Matlab, z pomocą narzędzia wbudowanego “nntool”.

1.2. Zestaw danych

Zestaw danych uczących został pobrany ze strony:

<http://archive.ics.uci.edu/ml/datasets/Acute+Inflammations>

Zawiera on 120 rekordów, każdy posiadający 6 cech oraz 2 werdykty.

Opis poszczególnych cech zestawu:

- a1 Temperatura pacjenta w zakresie od 35 do 42 stopni Celcjusza,
- a2 Objaw - występowanie nudności (tak / nie),
- a3 Objaw - ból w okolicy lędźwiowej (tak / nie),
- a4 Objaw - ciągła potrzeba oddawania moczu (tak, nie),
- a5 Objaw - Bóle przy mikcji (tak, nie),
- a6 Objaw - Pieczenie, swędzenie lub obrzęk cewki moczowej (tak, nie),
- d1 Decyzja - Zapalenie pęcherza moczowego (tak, nie),
- d2 Decyzja - Zapalenie nerek pochodzenia miedniczkowego (tak, nie).

1.3. Przygotowanie danych

Zrealizowana sieć ma za zadanie klasyfikację czy pacjent jest chory na jakąś z dwóch podanych chorób na podstawie posiadanych objawów. Występują cztery możliwości decyzji, pierwsza z nich określa pacjenta jako zdrowego. Druga oraz trzecia decyzja wskazuje na chorowanie na jedną z chorób. Decyzja czwarta pokazuje bardzo groźny przypadek chorowania na dwie z dostępnych w zbiorze chorób.

Podczas przygotowywania danych nie napotkano błędnych rekordów lub brakujących danych.

Wartości ‘tak’ lub ‘nie’ zamieniono na wartości binarne ‘1’ lub ‘0’. Przecinki zostały zamienione na kropki w celu przystosowania danych do skryptu w programie Matlab.

Następnie dokonana została normalizacja danych w celu ujednolicenia danych, które będziemy później przetwarzać. Do normalizacji użyto poniższego wzoru:

$$y_{norm} = \frac{(y_{max} - y_{min})(x - x_{min})}{(x_{max} - x_{min})} + y_{min} \quad (1.1)$$

gdzie,

y_{min} - minimalna wartość docelowa,

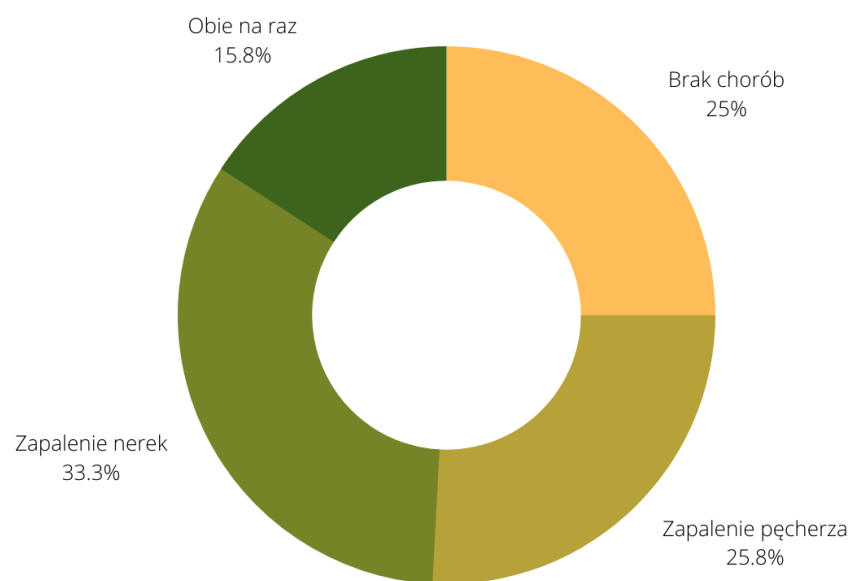
y_{max} - maksymalna wartość docelowa,

x_{min} - minimalna wartość w zbiorze,

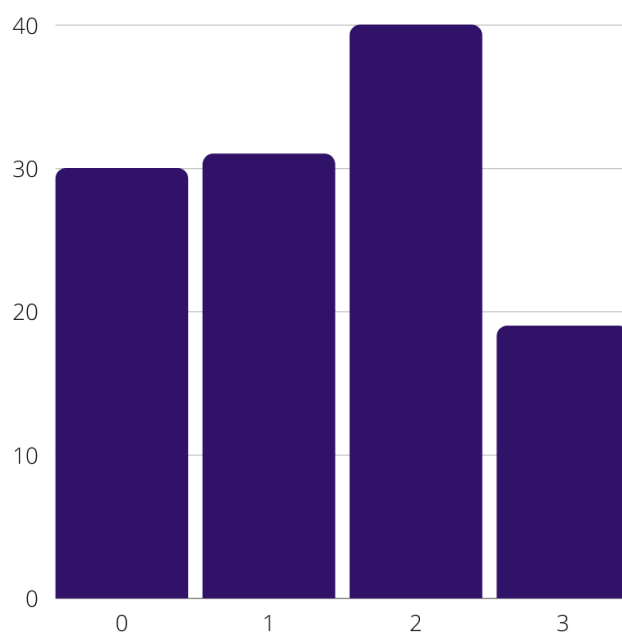
x_{max} - maksymalna wartość w zbiorze,

x - wartość przed normalizacją.

Po procesie normalizacji dane zostały przeanalizowane pod względem dużych rozbieżności w rozkładzie klas. Jak widać na rysunku (1.1) oraz (1.2) dane z klasy 0 (brak chorób) oraz 1 (zapalenie pęcherza) są zbliżone ilościowo do siebie. Rozbieżności o około 15% w porównaniu do klas 0 oraz 1, występują w klasach 3 (zapalenie nerek) oraz 4 (obie choroby jednocześnie). Mimo występujących rozbieżności klasy nie zostały okrojone. Decyzja ta spowodowana była bardzo małą liczbą rekordów w analizowanej bazie co mogłoby doprowadzić do niewystarczającej ilości danych uczących.



Rysunek 1.1: Procentowy rozkład klas w zbiorze.

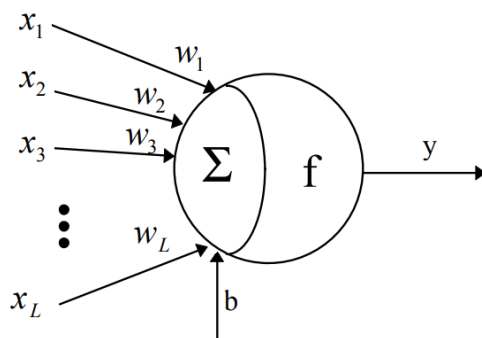


Rysunek 1.2: Ilościowy rozkład klas.

2. Wstęp teoretyczny

2.1. Model sztucznego neuronu

Sieć neuronowa składa się z pojedynczych neuronów połączonych ze sobą. Należy zatem zapoznać się z modelem pojedynczego neuronu w celu zrozumienia problemu sieci neuronowych.



Rysunek 2.3: Model neuronu [2].

Pojedynczy neuron posiada wejścia odpowiadające za odbieranie sygnału wejściowego do późniejszego przetworzenia. Każde z wejść ma współczynnik wagowy określający jak bardzo dane wejście wpływa na rezultat wynikowy danego neuronu. Dodatkowym elementem wejściowym jest “bias”, którego wartość jest równa dla wszystkich neuronów w danej warstwie. Całość z poprzednio podanych informacji do wejść neuronu trafia do sumatora gdzie odbywa się proces wyznaczania łącznego pobudzenia neuronu. Wartość pobudzenia trafia następnie do funkcji aktywacji w celu wyznaczenia sygnału wyjściowego neuronu. Wyjście to określane jest zależnością:

$$y = f\left(\sum_{j=1}^L w_j x_j + b\right) \quad (2.2)$$

gdzie,

y - wyjście neuronu,

w_j - waga j -tego wejścia,

x_j - j -ty sygnał wejściowy,

b - bias [1] [2].

2.2. Funkcja aktywacji

Neuron sigmoidalny posiada sigmoidalną unipolarną funkcję aktywacji, która przyjmuje wartości z zakresu (0,1), lub bipolarną funkcję aktywacji przyjmującą wartości z przedziału (-1, 1). W sieci utworzonej za pomocą programu Matlab wykorzystywana jest funkcja bipolarna.

$$f(x) = tgh(\beta x) = \frac{e^{\beta x} + e^{-\beta x}}{e^{\beta x} - e^{-\beta x}} = \frac{e^{2\beta x} - 1}{e^{2\beta x} + 1}. \quad (2.3)$$

Funkcja ta jest różniczkowalna, pochodna z funkcji bipolarnej wynosi:

$$\begin{aligned} f'(x) &= \frac{2\beta e^{2\beta x}(e^{2\beta x}+1) - 2\beta e^{2\beta x}(e^{2\beta x}-1)}{(e^{2\beta x}+1)^2} = \beta \frac{4e^{2\beta x}}{(e^{2\beta x}+1)^2} = \beta \frac{2}{e^{2\beta x}+1} \frac{2e^{2\beta x}}{e^{2\beta x}+1} = \\ &= \beta(1-f(x))(1+f(x)) = \beta(1-f^2(x)) [1][3]. \end{aligned} \quad (2.4)$$

2.3. Model sieci wielowarstwowej

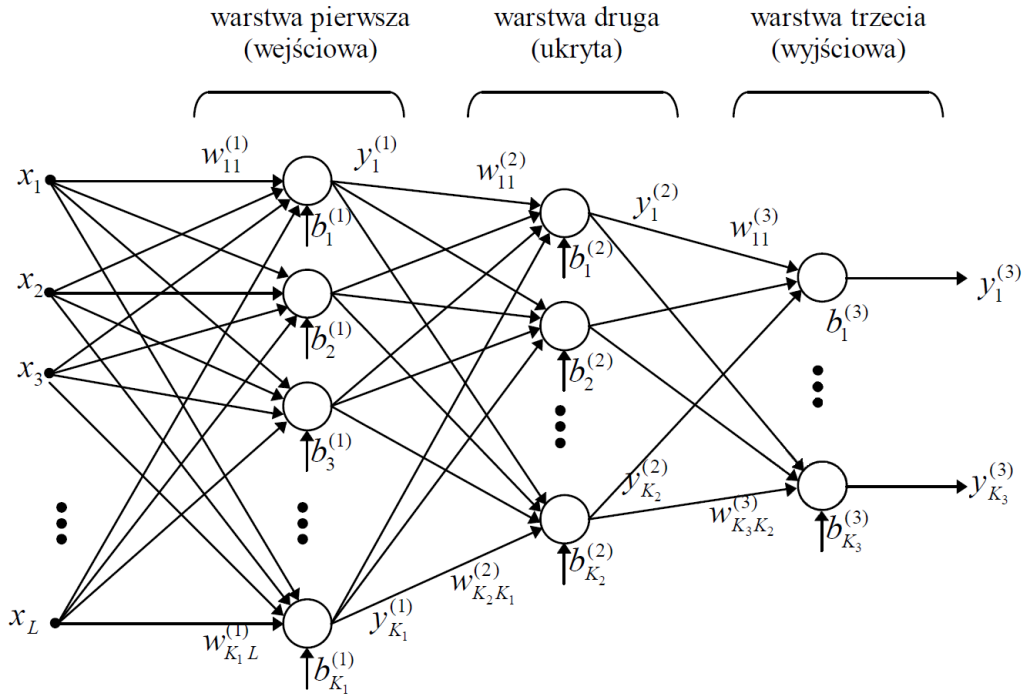
Neurony połączone ze sobą tworzą sieć neuronową. Każdy neuron z poprzedniej warstwy łączy się z neuronem warstwy następnej. Sygnały są przekazywane w kierunku wyjścia po wprowadzeniu zmian na poszczególnych neuronach. Kończącym efektem jest odpowiedź sieci w postaci pojedynczego sygnału w przypadku sieci o jednym wyjściu, lub odpowiedzią wielu sygnałową [1][4].

Każda z warstw sieci posiada macierz wag (w), macierz przesunięć (b), funkcję aktywacji (f) oraz wektor sygnałów na wyjściu warstwy (y). Działanie poszczególnych warstw opisuje poniższy wzór:

$$\mathbf{y}^{(n)} = \mathbf{f}^{(n)} \left(\mathbf{w}^{(n)} \mathbf{y}^{(n-1)} + \mathbf{b}^{(n)} \right). \quad (2.5)$$

Dla całego rozwiązania sieci z rysunku 2.4 wzór ten przyjmie postać:

$$\mathbf{y}^{(2)} = f^{(2)} \left(\mathbf{w}^{(2)} \mathbf{f}^{(1)} \left(\mathbf{w}^{(1)} \mathbf{f}^{(0)} \left(\mathbf{w}^{(0)} \mathbf{x} + \mathbf{b}^{(0)} \right) + \mathbf{b}^{(1)} \right) + \mathbf{b}^{(2)} \right) [4]. \quad (2.6)$$



Rysunek 2.4: Model sieci wielowarstwowej [4].

Mając wzór dla sygnału wyjściowego danego neuronu możemy przedstawić funkcję celu. Jest ona przydatna do określenia błędu popełnianego przez sieć. Funkcja celu dla sieci z modelu podanego na rysunku (2.4):

$$E = f^{(3)} \left(\sum_{i_2=1}^{K_2} w_{i_3 i_2}^{(3)} f^{(2)} \left(\sum_{i_1=1}^{K_1} w_{i_2 i_1}^{(2)} f^{(1)} \left(\sum_{j=1}^L w_{i_1 j}^{(1)} x_j + b_{i_1}^{(1)} \right) + b_{i_2}^{(2)} \right) + b_{i_3}^{(3)} \right) - \hat{y}_{i_4} \quad [1][4]. \quad (2.7)$$

Uproszczona postać funkcji celu:

$$E = \frac{1}{2} \sum_{i=1}^M e_i^2 \cdot [1][4] \quad (2.8)$$

2.4. Algorytm wstecznej propagacji błędu

Algorytm wstecznej propagacji błędu dominuje wśród metod uczenia sieci jednokierunkowych. Polega on na dobraniu odpowiednich wag tak aby sieć popełniała jak najmniejszy błąd na wyjściu. Istnieją dwa sposoby na adaptację wag. Pierwszy z nich to aktualizacja wag po przejściu wszystkich danych uczących drugi to zmiana wag przyrostowo wraz z nadchodzącymi danymi. W algorytmie wstecznej propagacji następuje odwrócenie kierunku przepływu sygnałów, a następnie obliczane są odpowiednie składniki gradientu funkcji celu dla poszczególnych neuronów w warstwach. Do obliczenia zmiany wagi korzystamy ze wzoru:

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}} \quad (2.9)$$

gdzie η to współczynnik uczenia [1][4].

Wartość danej wagi w kolejnej chwili czasu można zapisać w postaci:

$$w_{ij}(t+1) = w_{ij}(t) - \eta \frac{\partial E}{\partial w_{ij}} [1][4]. \quad (2.10)$$

Gradientu dla danej wagi nie da się policzyć bezpośrednio, dlatego obliczamy poszczególne pochodne cząstkowe. Dla warstwy wyjściowej składniki te będą miały postać:

$$\frac{\partial E}{\partial w_{ij}^{(2)}} = \frac{\partial E}{\partial f^{(2)}} \frac{\partial f^{(2)}}{\partial z_i^{(2)}} \frac{\partial z_i^{(2)}}{\partial w_{ij}^{(2)}} [1][4]. \quad (2.11)$$

Gdzie odpowiednio każdy składnik można obliczyć, otrzymując zależność:

$$\frac{\partial E}{\partial w_{ij}^{(2)}} = (y_i - d_i) \frac{\partial f^{(2)}(z_i^{(2)})}{\partial z_i^{(2)}} v_j^{(1)} [1][4]. \quad (2.12)$$

W podobny sposób oblicza się wartość gradientu dla warstwy ukrytej :

$$\begin{aligned}
\frac{\partial E}{\partial w_{ij}^{(1)}} &= \frac{\partial E}{\partial f^{(2)}} \frac{\partial f^{(2)}}{\partial z_i^{(2)}} \frac{\partial z_i^{(2)}}{\partial f^{(1)}} \frac{\partial f^{(1)}(z_j^{(1)})}{\partial z_j^{(1)}} \frac{\partial z_j^{(1)}}{\partial w_{ij}^{(1)}} = \\
&= \sum_{i=1}^M (y_i - d_i) \frac{\partial f^{(2)}(z_i^{(2)})}{\partial z_i^{(2)}} w_{ij}^{(2)} \frac{\partial f^{(1)}(z_j^{(1)})}{\partial z_j^{(1)}} v_i^{(0)} [1][4].
\end{aligned} \tag{2.13}$$

Wartość gradientu dla warstwy wejściowej:

$$\begin{aligned}
\frac{\partial E}{\partial w_{ij}^{(0)}} &= \frac{\partial E}{\partial f^{(2)}} \frac{\partial f^{(2)}}{\partial z_i^{(2)}} \frac{\partial z_i^{(2)}}{f^{(1)}} \frac{\partial f^{(1)}(z_j^{(1)})}{\partial z_j^{(1)}} \frac{\partial z_j^{(1)}}{\partial f^{(0)}} \frac{\partial f^{(0)}(z_i^{(0)})}{\partial z_i^{(0)}} \frac{\partial z_i^{(0)}}{\partial w_{ij}^{(0)}} = \\
&= \sum_{i=1}^M (y_i - d_i) \frac{\partial f^{(3)}(z_i^{(2)})}{\partial z_i^{(2)}} \sum_{k=1}^K w_{ij}^{(2)} \frac{\partial f^{(1)}(z_j^{(1)})}{\partial z_j^{(1)}} w_{ij}^{(1)} \frac{\partial f^{(0)}(z_i^{(0)})}{\partial z_i^{(0)}} x_j
\end{aligned} \tag{2.14}$$

3. Realizacja sieci neuronowej

3.1. Opis skryptu

Pracując nad skrypcem zdecydowano się na podzielenie go na dwie części. Pierwsza część odpowiada za badanie wpływu różnych ustawień ilości neuronów na poprawność klasyfikacji oraz błąd SSE. Drugi skrypt odpowiada za analizę wpływu współczynnika uczenia na poprawność klasyfikacji. Główną przesłanką do rozdzielenia skryptu było zmniejszenie złożoności kodu, co za tym idzie zwiększenie jego czytelności. Dodatkową zaletą podziału skryptu jest przyspieszenie kilkukrotnie czasu nauki danej sieci co pozwoliło przeprowadzić więcej eksperymentów oraz szybciej analizować wyniki sieci.

3.2. Skrypt dla zależności poprawności klasyfikacji od liczby neuronów w warstwach

```
1 close all
2 clear
3 load D:\STUDIA\Sem4\SztucznaInteligencja\Projekt\diagnosis_n
4
5 max_epoch = 14000;
6 err_goal = .5^2/length(T);
7 max_fail = 10000;
8
9 Ptest = zeros(6,18); % przechowuje dane testowe
10 Ttest = zeros(1,18); % przechowuje cele testowe
11 results = zeros([1,36]); % przechowuje wartosci pk
12 errors = zeros([1,36]); % przechowuje wartosci SSE
13
14 counter = 1;
15 lr = 1e-3;
16
17 % liczba neuronow w pierwszej warstwie ukrytej
18 for S1 = [2:1:7]
19     % liczba neuronow w drugiej warstwie ukrytej
20     for S2 = [1:1:6]
21         if S1 > S2
22             clear net tr o Ptest Ttest
23             % definicja perceptronu
24             net = feedforwardnet([S1, S2], 'traingd');
25             net.trainParam.epochs = max_epoch;
26             net.trainParam.goal = err_goal;
27             net.trainParam.lr = lr;
28             net.trainParam.max_fail = max_fail;
29             net.trainParam.showWindow = true;
30             net.divideParam.trainRatio=0.85;
31             net.divideParam.valRatio=0;
32             net.divideParam.testRatio=0.15;
33             [net,tr] = train(net,Pns,Ts);
```

```

34         %wyciagniecie danych testowych wylosowanych ze
        zbioru
35         Ptest = Pn(:,tr.testInd);
36         Ttest = T(:,tr.testInd);
37
38         %wyciagniecie rezultatu poprawnosci klasyfikacji
        oraz SSE
39         o = net(Ptest);
40         errors(counter) = sse(net, Ttest, o);
41         results(counter) = (1 - sum(abs(Ttest - o) >= 0.5)
        / length(Ttest)) * 100;
42         end
43         counter = counter + 1;
44     end
45 end
46
47 S1 = [2, 3, 4, 5, 6, 7];
48 S2 = [1, 2, 3, 4, 5, 6];
49
50 % przygotowanie otrzymanych po nauce sieci danych w celu
        narysowania odpowiednich wykresow
51 [S1X, S2Y] = meshgrid(S1, S2);
52 res = reshape(results, 6, 6);
53 err = reshape(errors, 6, 6);
54
55 % rysownie wykresu dla poprawno ci klasyfikacji
56 figure(100); mesh(S1X, S2Y, res)
57 xlabel('S1');
58 ylabel('S2');
59 zlabel('poprawnosc klasyfikacji');
60
61 % rysownie wykresu dla SSE
62 figure(101); mesh(S1X, S2Y, err)
63 xlabel('S1');
64 ylabel('S2');
65 zlabel('SSE');

```

Listing 1: Skrypt dla zmiennych S1 oraz S2.

Na początku skryptu definiowane są: błąd maksymalny, maksymalna ilość epok oraz współczynnik lr (learning rate). Dodatkowo zaimplementowane zostały cztery tablice, dwie odpowiadające za przechowywanie danych testujących dla sieci. Pozostałe dwie odpowiedzialne są za zbieranie poprawności klasyfikacji oraz błędu SSE w celu późniejszej wizualizacji wyników na wykresie.

W linii 19 oraz 21 zrealizowane zostały warstwy ukryte dla zmieniających się $S1$ oraz $S2$, czyli liczby neuronów w warstwie. Korzystając z zalecenia znajdującego się w instrukcji projektowej dodany został warunek uczenia się sieci dla liczby neuronów w warstwie pierwszej ukrytej większej od liczby neuronów w drugiej warstwie ukrytej.

Następnie zerowane są wartości mogące być pozostałościami po poprzedniej iteracji programu oraz definiowane na nowo ustawienia dla sieci z nową konfiguracją neuronów.

Kolejnym ostatnim elementem jest wydzielenie danych na podstawie obiektu sieci 'tr' a dokładnie atrybutu testInd, wartości wybranych przez skrypt jako dane testujące. Atrybut ten przechowuje indeksy w zbiorze wszystkich danych, które zostały podzielone według wcześniejszych założeń z linii 31-33.

Przed ostatnim elementem jest ustalenie dla danej konfiguracji sieci jej poprawności klasyfikacji oraz błędu SSE na podstawie danych testujących.

Zakończeniem skryptu jest wyrysowanie wykresów trójwymiarowych dla poprawności klasyfikacji w zależności od konfiguracji $S1$ oraz $S2$, a także błędu SSE dla takiej konfiguracji.

3.3. Skrypt dla zależności poprawności klasyfikacji od wartości współczynnika uczenia

```
1 close all
2 clear
3 load D:\STUDIA\Sem4\SztucznaInteligencja\Projekt\diagnosis_n
4
5 max_epoch = 14000;
6 err_goal = .5^2/length(T);
7 max_fail = 10000;
8
9 Ptest = zeros(6,18);      % przechowuje dane testowe
10 Ttest = zeros(1,18);     % przechowuje cele testowe
11 results = zeros([1,36]); % przechowuje warto ci poprawno ci
    klasyfikacji
12 errors = zeros([1,36]); % przechowuje warto ci b d w SSE (
    Sum Squared Error)
13
14 counter = 1;
15 S1 = 10;
16 S2 = 6;
17
18 for lr = [1e-1, 1e-2, 1e-3, 1e-4, 1e-5]
19     clear net tr o Ptest Ttest
20     % definicja percepcoru
21     net = feedforwardnet([S1, S2], 'traingd');
22     net.trainParam.epochs = max_epoch;
23     net.trainParam.goal = err_goal;
24     net.trainParam.lr = lr;
25     net.trainParam.max_fail = max_fail;
26     net.trainParam.showWindow = true;
27     net.divideParam.trainRatio=0.85;
28     net.divideParam.valRatio=0;
29     net.divideParam.testRatio=0.15;
30     [net,tr] = train(net,Pns,Ts);
31
32     %wyciagniecie danych testowych wylosowanych ze zbioru
33     Ptest = Pn(:,tr.testInd);
34     Ttest = T(:,tr.testInd);
35
36     %wyciagniecie rezultatu poprawnosci klasyfikacji oraz SSE
37     o = net(Ptest);
38     errors(counter) = sse(net, Ttest, o);
39     results(counter) = (1 - sum(abs(Ttest - o) >= 0.5) /
length(Ttest)) * 100;
40     counter = counter + 1;
41 end
42
43 % rysownie wykresu dla poprawno ci klasyfikacji
44 LrRange = [1e-1, 1e-2, 1e-3, 1e-4, 1e-5];
45 figure(21);plot(LrRange, results);
46 grid;
47 axis([0 0.011 0 inf]);
48 title('Zale no poprawno ci klasyfikacji od wsp czynnika
lr');
```



```

49 xlabel('Lr');
50 ylabel('poprawno    klasyfikacji [%]');
51
52 % rysownie wykresu dla poprawno ci klasyfikacji
53 figure(20); plot(LrRange, errors);
54 grid;
55 axis([0 0.011 0 inf]);
56 title('Zale no    SSE od wsp  czynnika lr');
57 xlabel('Lr');
58 ylabel('SSE');

```

Listing 2: Skrypt dla zmiennego współczynnika lr.

Skrypt drugi nie różni się zbyt wiele od skryptu poprzedniego lecz zawiera parę elementów przystosowujących go do danego problemu jakim jest badanie poprawności klasyfikacji i błędu SSE dla zmiennego współczynnika lr.

Pierwszym z elementów jest zmiana deklaracji lr na ustawienie na stałe wartości S1 oraz S2.

Kolejną widoczną zmianą jest zrezygnowanie z jednej pętli for i zmiana pozostałej na podmianę wartości lr.

Ostatnim elementem odróżniającym ten skrypt od poprzedniego jest sposób rysowania wykresów. Tym razem mamy do czynienia z wykresem dwuwymiarowym gdyż badamy zależność PK (oraz SSE) od parametru LR. Dla ulepszenia widoczności na wykresach zastosowano funkcje axis() pozwalającą na określenie przestrzeni wykresu widocznej dla użytkownika. Jest to kluczowe aby można było dobrze przeanalizować otrzymane wyniki.

4. Eksperymenty

Eksperymenty z racji podziału skryptu zależne są od użytego skryptu. Tak więc pierwsze eksperymenty dotyczyć będą zależności ilości neuronów w poszczególnych warstwach ukrytych na poprawność klasyfikacji, a eksperymenty z drugiej części zależności pk od zmiennego współczynnika lr.

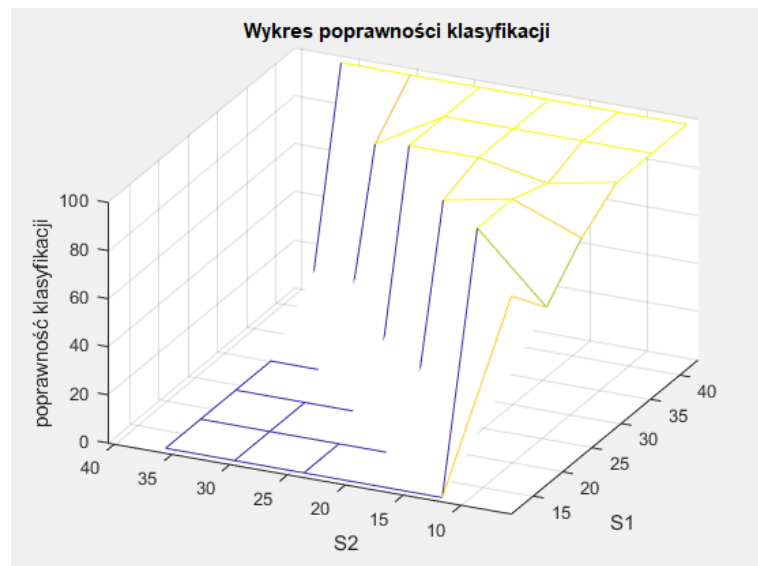
4.1. Eksperyment 1

Eksperyment pierwszy został przeprowadzony dla przykładowych liczb neuronów będących wielokrotnościami 6.

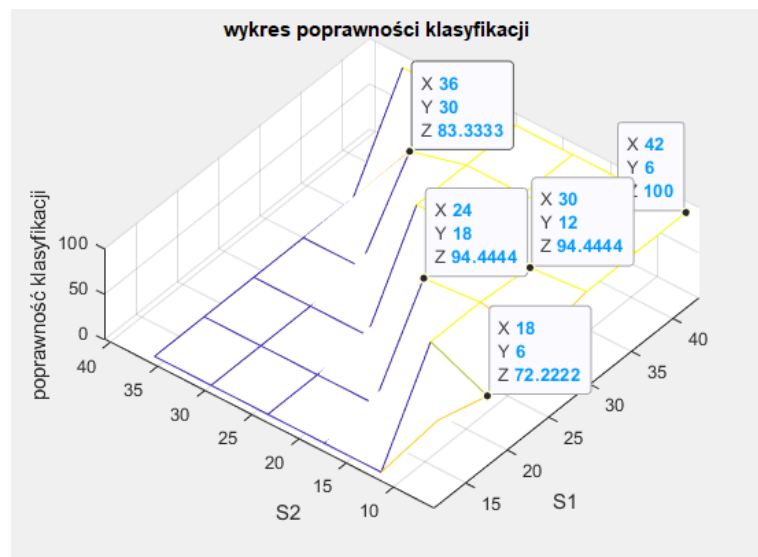
Dla warstwy pierwszej ukrytej ustalony został zakres [12:6:42], dla warstwy drugiej ukrytej zakres liczby neuronów to [6:6:36]. Współczynnik uczenia określony został na $1e^{-4}$, a liczba epok ustalona na 7000.

Po wielu niezapisanych w tym dokumencie eksperymentach liczbę epok zmieniano od 40 000 aż do wyżej opisanych 7000. Powodem tych zmian było bardzo dobre uczenie się sieci dla wyższych ilości epok co dawało bardzo schematyczne wyniki ok 98-100 mod poprawności klasyfikacji. Wyniki te były mało interesujące, gdyż nie było można stwierdzić żadnych zależności dla otrzymanych wartości. Dodatkowo można było zauważyć przeuczanie się sieci co nie jest wskazane.

Ustalając podane wyżej wartości współczynników sieci uruchomiony został skrypt. Wyniki dla poprawności klasyfikacji widoczne są na rysunkach (4.5) oraz (4.6), zaś wyniki dla błędu SSE widoczne są na wykresach z rysunków (4.7) oraz (4.8).

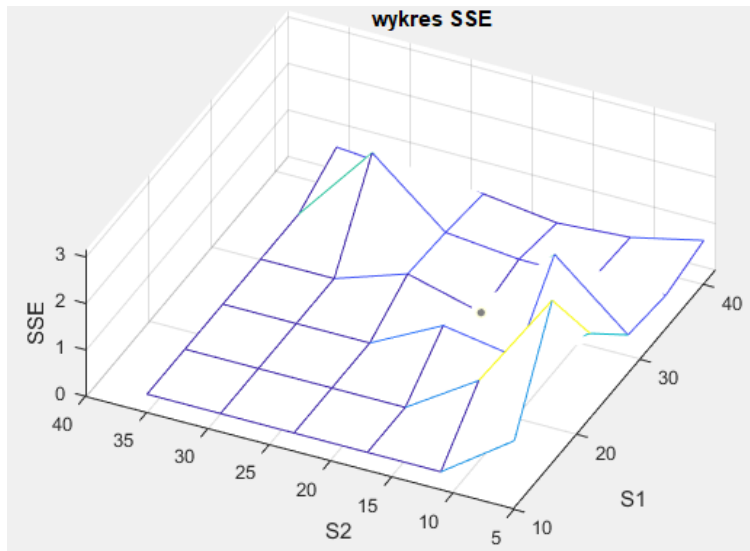


Rysunek 4.5: Wykres poprawności klasyfikacji dla eksperymentu 1.

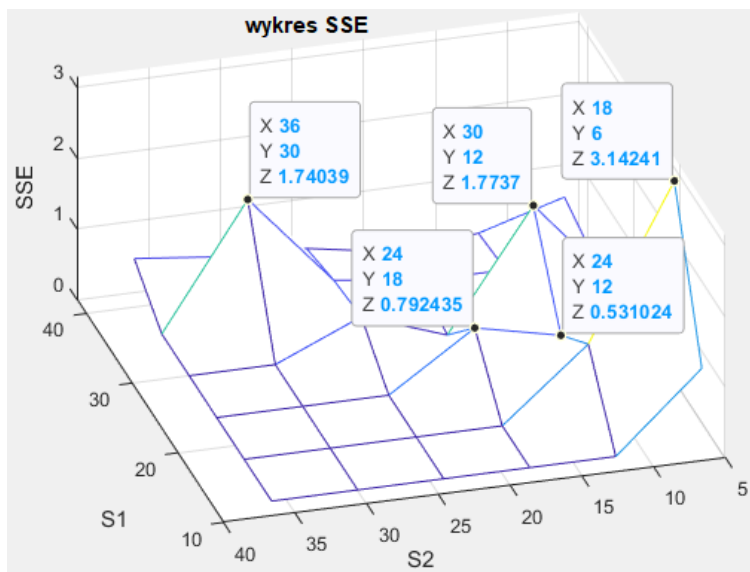


Rysunek 4.6: Wykres z rysunku 4.5 z dodatkowymi punktami.

Analizując wyniki otrzymane w eksperymencie zauważamy dobry poziom poprawności klasyfikacji dla większości konfiguracji neuronów w sieci. Rozpiętość poprawności klasyfikacji dla tego eksperymentu wynosiła od 72.22% do 100%. Jak poprawność klasyfikacji na poziomie 94-100% jest bardzo pozytywną odpowiedzią sieci na zadany jej problem klasyfikacji, tak widać wyraźne problemy sieci w przypadku punktu $S1 = 18$ $S2 = 6$. W celu dodatkowej weryfikacji otrzymanych wyników przygotowany został wykres SSE.



Rysunek 4.7: Wykres SSE dla eksperymentu 1.

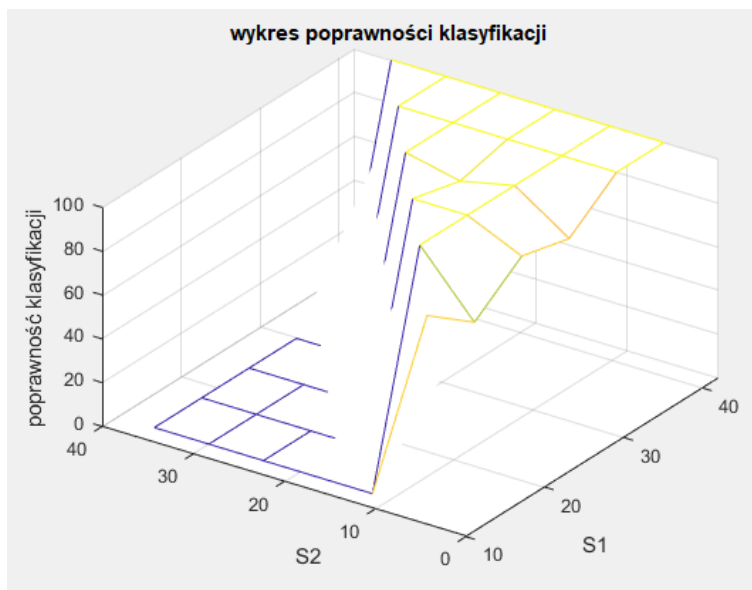


Rysunek 4.8: Wykres z rysunku 4.7 z dodatkowymi punktami.

Jak widać w punkcie charakterystycznym dla poprzedniego wykresu możemy zauważyć że SSE ma największą wartość równą 3.14, co w przypadku małej ilości danych wejściowych jest dosyć dużym błędem. Obserwując pozostałe wyniki wydaje się że punkt ten bardzo odstaje od pozostałych i prawdopodobnie była to zmiana losowa zależna od wylosowanych wag na początku uczenia sieci o podanej konfiguracji. W celu weryfikacji czy błąd ten występuje dla podanej konfiguracji w tym punkcie sporządzono kolejny eksperyment dla tych samych ustawień sieci. Eksperyment ten opisany został w punkcie "4.2. Eksperyment 2".

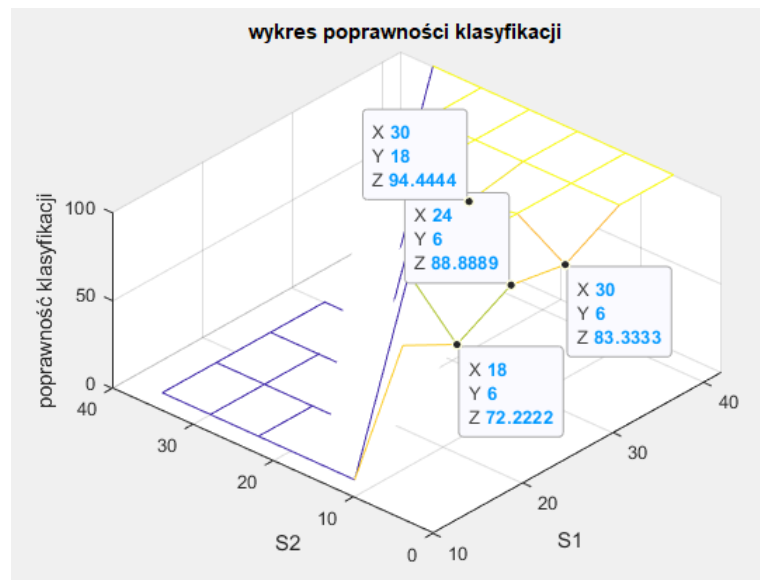
4.2. Eksperyment 2

Celem eksperymentu będzie określenie czy błąd występujący w punkcie $S1 = 18$ $S2 = 6$ jest błędem występującym stale dla tej konfiguracji czy tylko dziełem przypadku. Przeprowadzony zatem został eksperyment z takimi samymi wartościami współczynników jak poprzednio. Wyniki otrzymane podczas uczenia sieci można zaobserwować na wykresach (4.9) oraz (4.10).

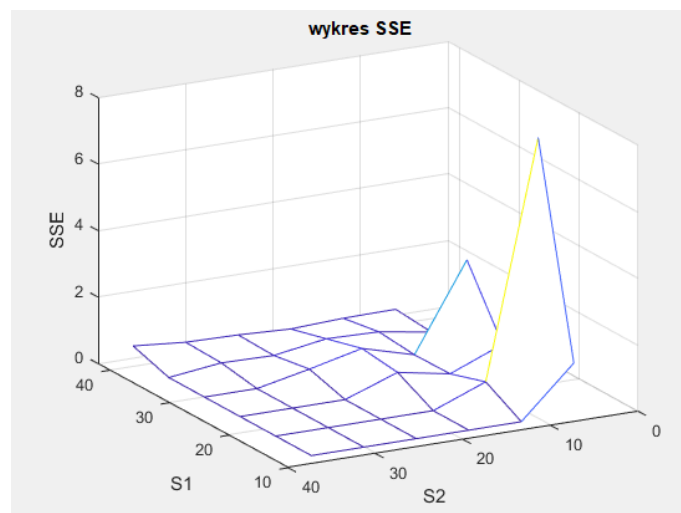


Rysunek 4.9: Wykres poprawności klasyfikacji dla eksperymentu 2.

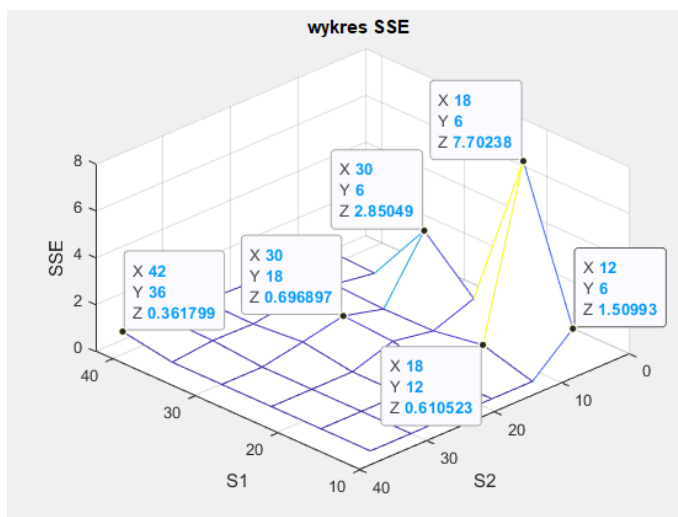
Analizując otrzymane dane widzimy, iż spadek poprawności klasyfikacji dla interesującego nas punktu nadal występuje. Można zatem przyjąć iż jest to spowodowane konfiguracją ilości neuronów. Warto również pamiętać że ilość eksperymentów nie pozwala stwierdzić postawionej hipotezy jednoznacznie, lecz dla celów projektowych uznane zostało takie założenie.



Rysunek 4.10: Wykres z rysunku 4.9 z dodatkowymi punktami.



Rysunek 4.11: Wykres SSE dla eksperymentu 2.



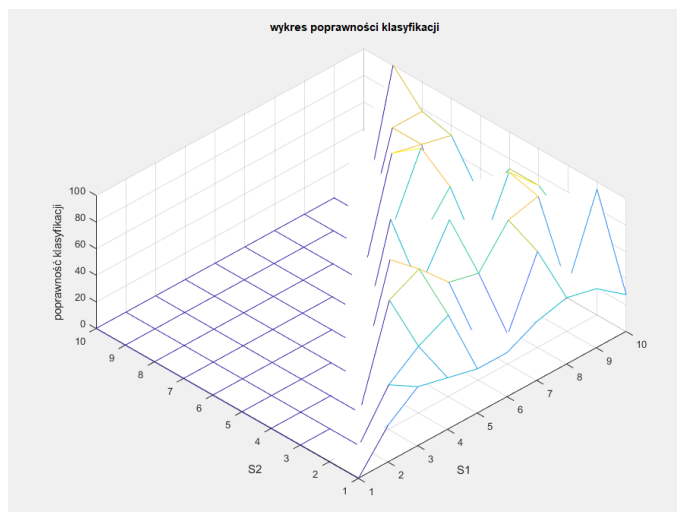
Rysunek 4.12: Wykres SSE z rysunku 4.11 z dodatkowymi punktami.

Jak widać na otrzymanym wykresie SSE (rysunek 4.11) sieć dla tego podejścia nauczyła się trochę lepiej niż dla podejścia z eksperymentu 1. Mimo to, punkt (18,6) nadal generuje gorszą poprawność klasyfikacji sieci.

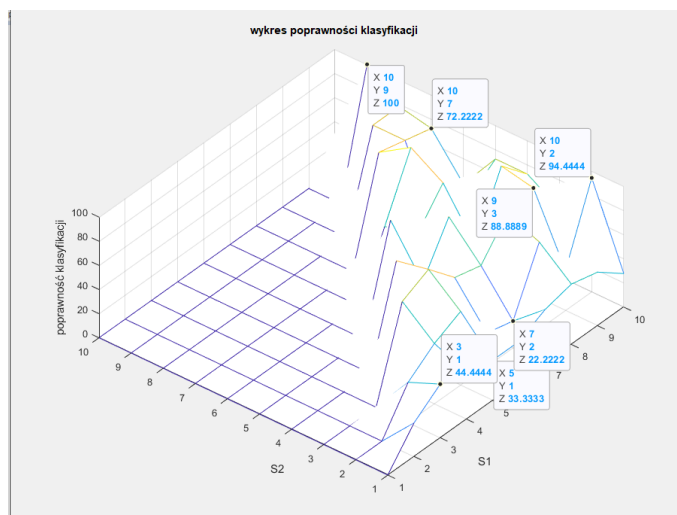
Eksperyment pozwolił na określenie czy wcześniej wymieniony punkt jest elementem dla, którego sieć standardowo odpowiada gorzej. Założenia potwierdziły się, dlatego kolejne eksperymenty będą kontunowane dla innych konfiguracji sieci w celu dogłębnniejszego jej zbadania.

4.3. Eksperyment 3

Kolejnym eksperymentem dla różnych konfiguracji liczby neuronów jest eksperyment polecony przez prowadzącego. Celem tego badania ma być analiza działania sieci neuronowej dla liczby neuronów od 1 do 10 dla warstw ukrytych. W tym celu zmienione zostały wartości S1 oraz S2, pozostałe współczynniki zostały nie zmienione.

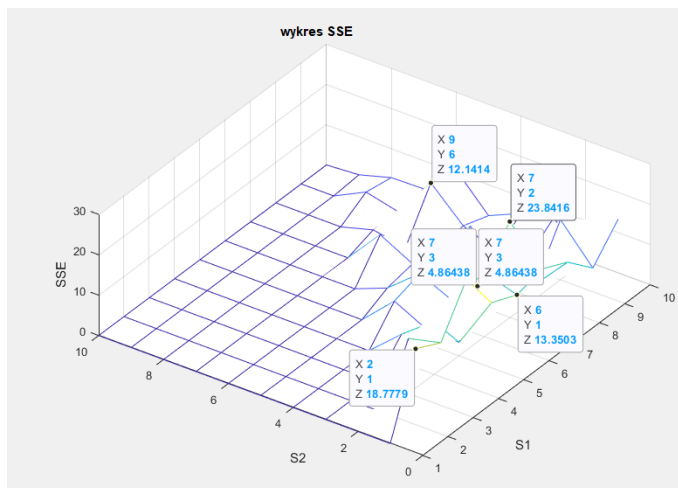


Rysunek 4.13: Wykres poprawności klasyfikacji dla eksperymentu 3.

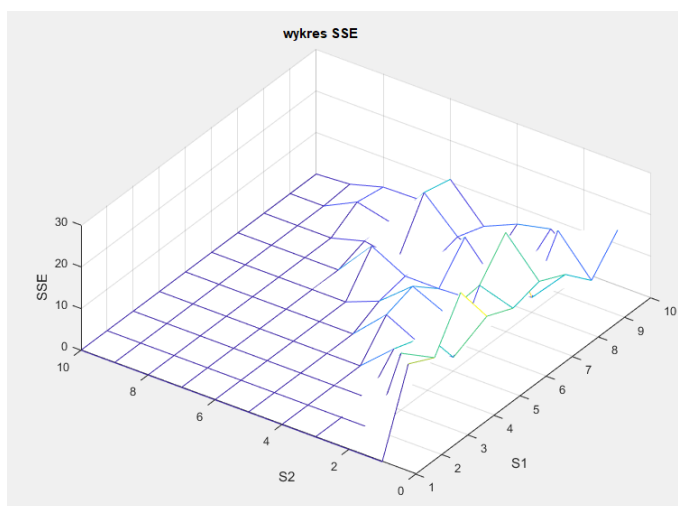


Rysunek 4.14: Wykres poprawności klasyfikacji dla eksperymentu 3 z dodatkowymi punktami.

Jak widać na rysunkach (4.13) oraz (4.14) sieć dla mniejszej liczebności neuronów w warstwach ukrytych dla danej konfiguracji sieci zachowuje się dosyć chaotycznie. Spowodowane jest to prawdopodobnie niedostateczną liczbą epok, która dla tego eksperymentu wynosiła 7000. W celu weryfikacji postawionej hipotezy przeprowadzony został eksperyment 4.



Rysunek 4.15: Wykres SSE dla eksperymentu 3.

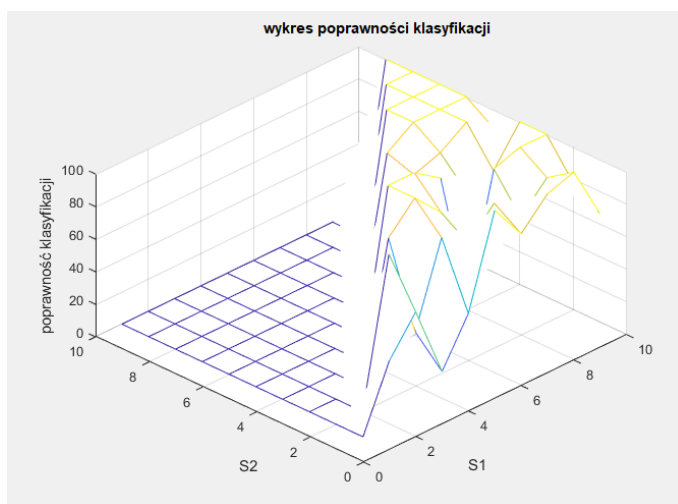


Rysunek 4.16: Wykres SSE dla eksperymentu 3 z punktami.

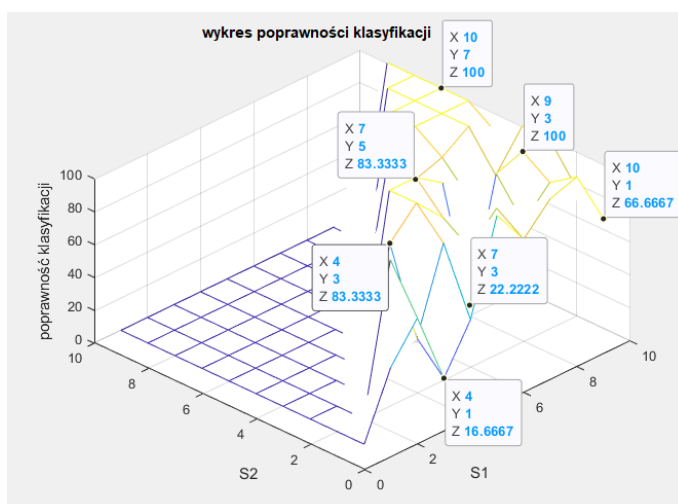
Analizując dane z otrzymanych wykresów widać bardzo niski stopień poprawności klasyfikacji dla małych ilości neuronów. Dla sieci w których był tylko jeden neuron w drugiej warstwie ukrytej widać bardzo podobne wartości. Im więcej neuronów zawierała sieć tym lepsze otrzymywano wyniki klasyfikacji. Największy procent poprawności klasyfikacji otrzymano dla $S1 = 10$ oraz $S2 = 9$ czyli największej możliwej ilości neuronów dla tego eksperymentu.

4.4. Eksperyment 4

Podczas analizy eksperymentu 3 zwrócono uwagę na prawdopodobnie nie wystarczającą ilość epok dla sieci neuronowych z ilościami neuronów w warstwach ukrytych od 1 do 10. Dążąc do rozwiązania problemu przygotowany został eksperyment dla tej samej konfiguracji pozostałych współczynników co w eksperymencie 3 lecz z kilkukrotnie większą ilością iteracji (epok). Dla 30 000 epok wyniki sieci wyglądają tak jak pokazano na rysunku (4.17) oraz (4.18).



Rysunek 4.17: Wykres poprawności klasyfikacji dla eksperymentu 4.



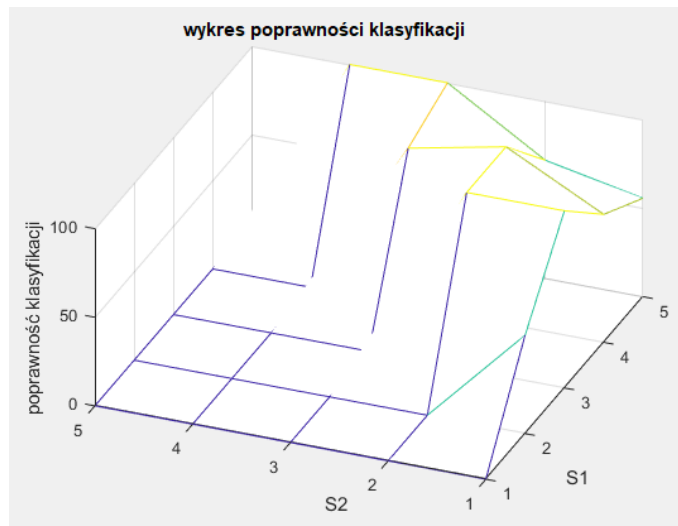
Rysunek 4.18: Wykres dla eksperymentu 4 z punktami.

Jak widać sieć mocno zwiększyła swoją poprawność klasyfikacji w porównaniu do poprzedniczki. Jednak widać nadal rozchwanie wyników co wskazuje iż liczba epok 30 tysięcy jest nadal nie dostatecznie duża dla danego współczynnika lr oraz małej ilości neuronów w warstwach.

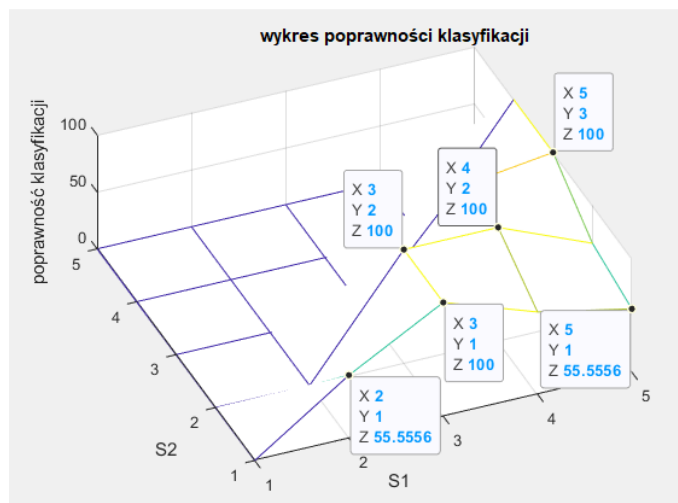
Widoczny również jest spadek poziomu poprawności klasyfikacji w miejscach gdzie zakres liczby neuronów w warstwie pierwszej ukrytej badanej sieci wynosi 1-4. Powodem takiego zachowania sieci jest prawdopodobnie zbyt krótki okres uczenia się sieci. W kolejnym eksperymencie sprawdzona zostanie poprawność wniosków wyciągniętych z analizy powyższych wyników.

4.5. Eksperyment 5

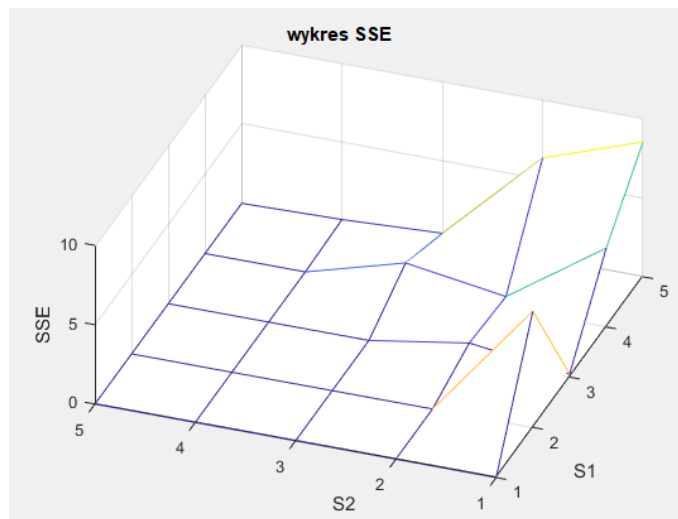
Ten eksperyment jest ostatnim punktem eksperymentów i analiz wyników dla zmiennej liczby neuronów w warstwach ukrytych sieci. Badana sieć została okrojona do sieci uczącej się dla zakresu liczby neuronów od 1 do 5 dla obu warstw. Liczba epok została zwiększona do 50 tysięcy. Spodziewanym wynikiem będzie zwiększenie się poprawności klasyfikacji dla badanej sieci.



Rysunek 4.19: Wykres SSE dla eksperymentu 5.



Rysunek 4.20: Wykres SSE dla eksperymentu 5 z punktami.

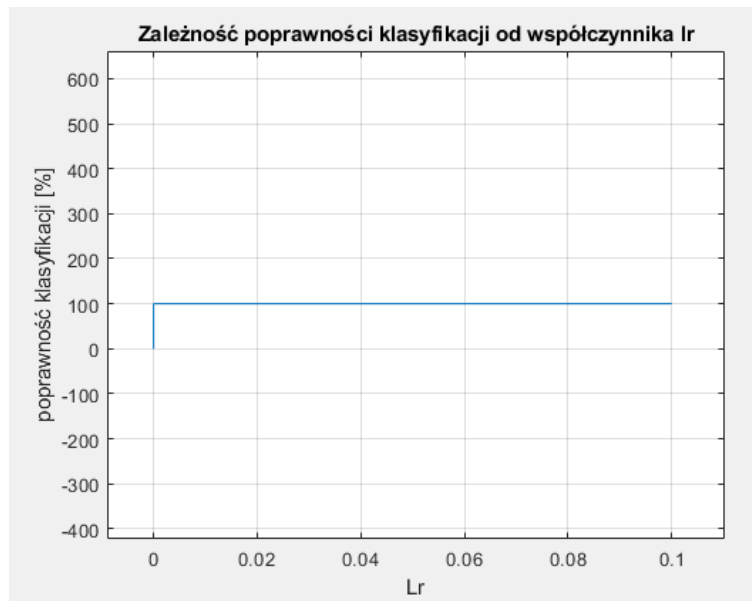


Rysunek 4.21: Wykres SSE dla eksperymentu 5 z punktami.

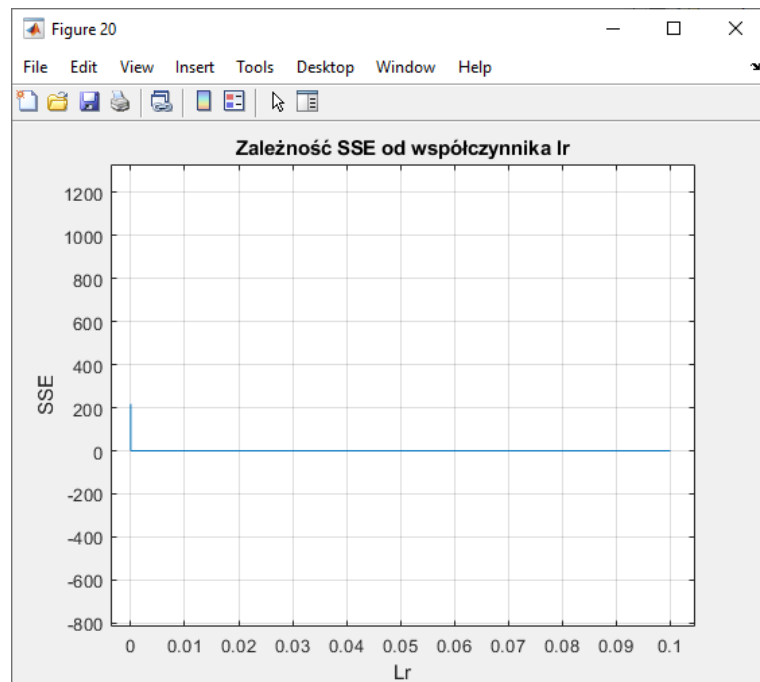
Jak widać na wykresach z rysunków (4.19) oraz (4.20) poprawność tak jak wcześniej przewidziano, poprawiła się. Eksperyment ten zakończył serię badań nad działaniem liczby neuronów dla warstw ukrytych na poziom uczenia sztucznej sieci neuronowej. Szerzej opisane wnioski z eksperymentów od 1 do 5 zostały zawarte na końcu pracy projektowej razem z wnioskami z eksperymentów 6-10.

4.6. Eksperyment 6

Eksperyment ten zaczyna serię badań wpływu współczynnika lr na poziom nauczania sieci neuronowej. Sieć uczona będzie dla $S1 = 36$ $S2 = 24$ oraz dla liczby epok równej jak w poprzednim eksperymencie 50000. Liczba neuronów została wybrana na podstawie pierwszego eksperymentu dla którego ta liczba neuronów wskazała 100 poprawność klasyfikacji sieci. Współczynnik lr będzie zmieniany od wartości $1e-1$ do $1e-10$.



Rysunek 4.22: Wykres SSE dla eksperymentu 6.

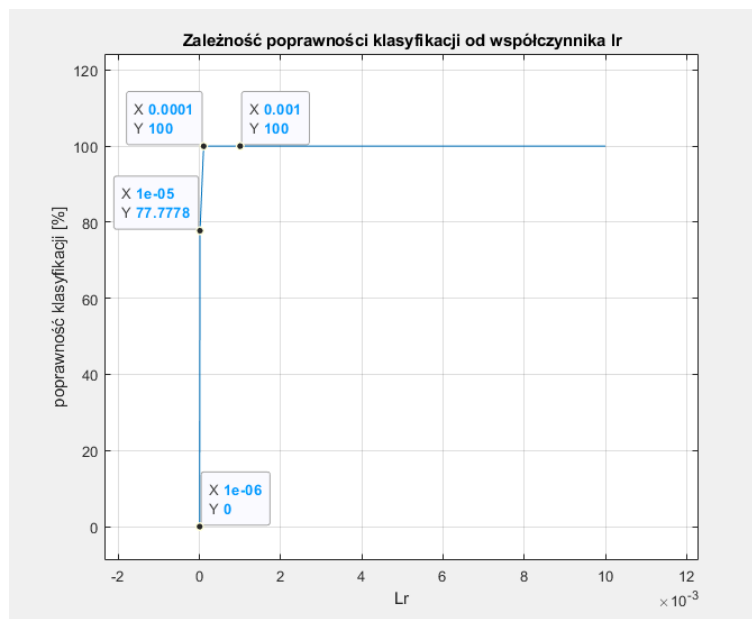


Rysunek 4.23: Wykres poprawności klasyfikacji dla eksperymentu 6.

Otrzymane wykresy z rysunków (4.22) oraz (4.23) pokazują iż sieć nauczyła się w 100 % dla większości współczynników lr . Jednak jest to bardzo niefortunny przypadek do omawiania wpływu współczynnika lr na poziom nauczania. Co prawda widać iż dla bardzo małego lr równego $1e-10$ poprawność klasyfikacji zbliżyła się do zera, jednak nie można określić wpływu współczynnika w innych miejscach wykresu. W tym celu przeprowadzony został eksperyment 7

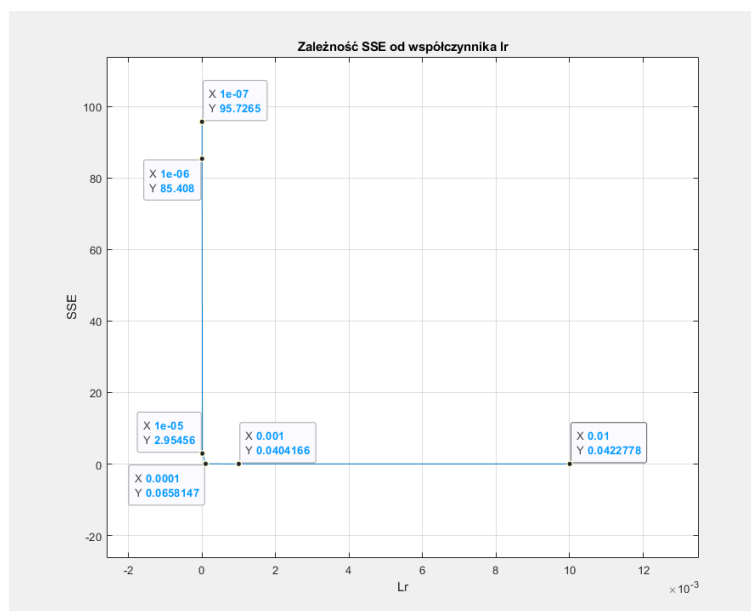
4.7. Eksperyment 7

Dla sprawdzenia zachowania sieci przy zmniejszonej ilości epok przeprowadzony został następujący eksperyment. Liczba neuronów w warstwach ukrytych $S1 = 36$, $S2 = 24$ tak jak poprzednio. Liczba epok zmniejszona pięciokrotnie w stosunku do eksperymentu 6, a więc ustawione zostało 10 000 epok. Zmniejszony został również zakres zmiany LR, usunięto $1e-1$.



Rysunek 4.24: Wykres poprawności klasyfikacji dla eksperymentu 7.

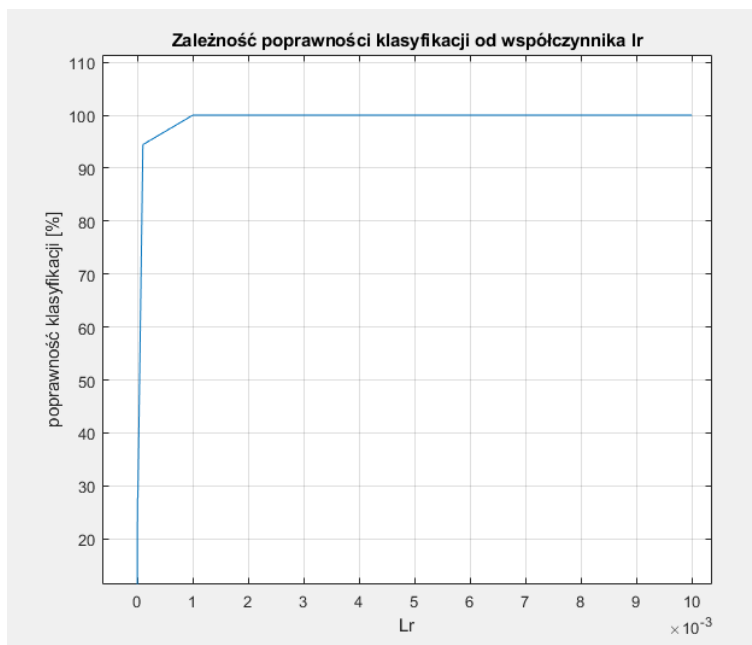
Analizując wyniki sieci zauważamy niewielką zmianę w nachyleniu wykresu między punktem dla $lr = 1e-5$ a punktem $lr = 1e-4$. Wyniki te sugerować mogą występowanie wpływu współczynnika lr na poziom nauczania sieci, lecz dla dogłębniejszego zbadania tego problemu przeprowadzone zostały kolejne eksperymenty.



Rysunek 4.25: Wykres SSE dla eksperymentu 7

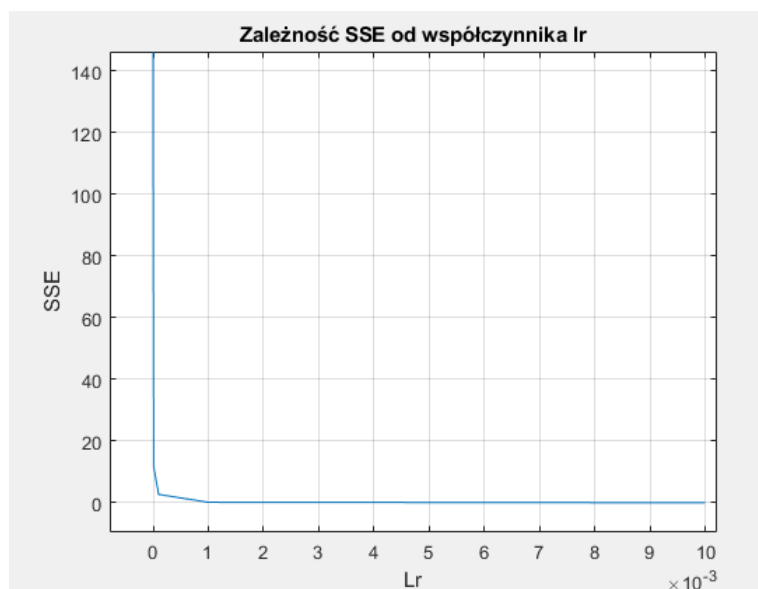
4.8. Eksperyment 8

W tym eksperymencie mocno ograniczona została liczba epok. W celu zbadania jak zachowa się sieć dla ograniczonego czasu uczenia w przypadku różnych współczynników LR Liczba epok została ustalona na poziomie 800 iteracji.



Rysunek 4.26: Wykres poprawności klasyfikacji dla eksperymentu 8.

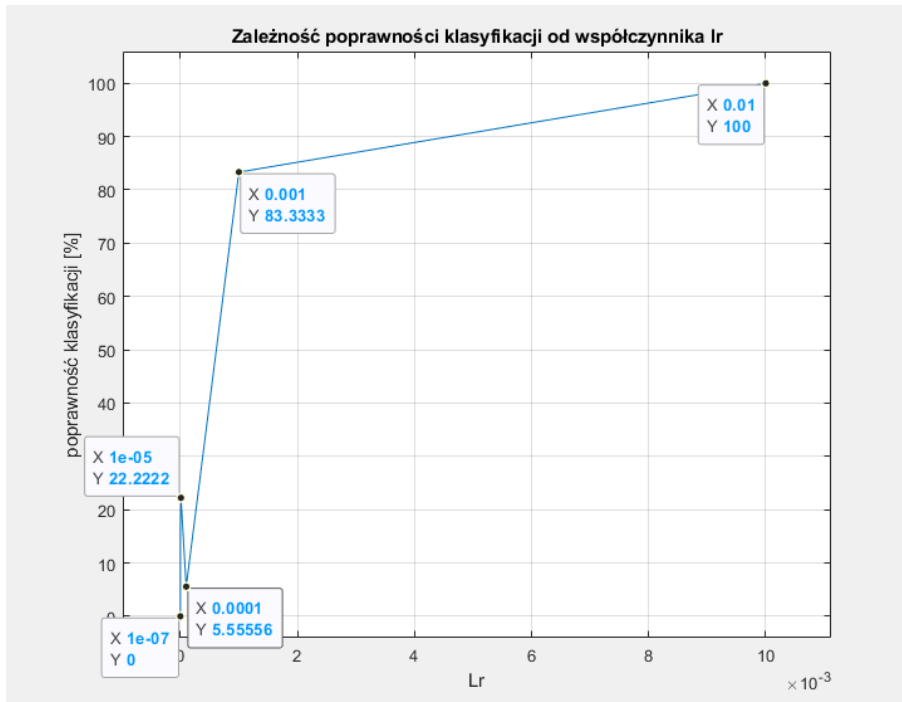
Wyniki pokazują zmianę poziomu poprawności klasyfikacji. Tym razem nastąpiło większe odchylenie od poziomu z eksperymentu 1 co pokazuje, iż współczynnik lr ma wpływ na uczenie sieci. Kontynuując badania nad wpływem współczynnika lr przeprowadzono eksperymenty 9 oraz 10.



Rysunek 4.27: Wykres SSE dla eksperymentu 8.

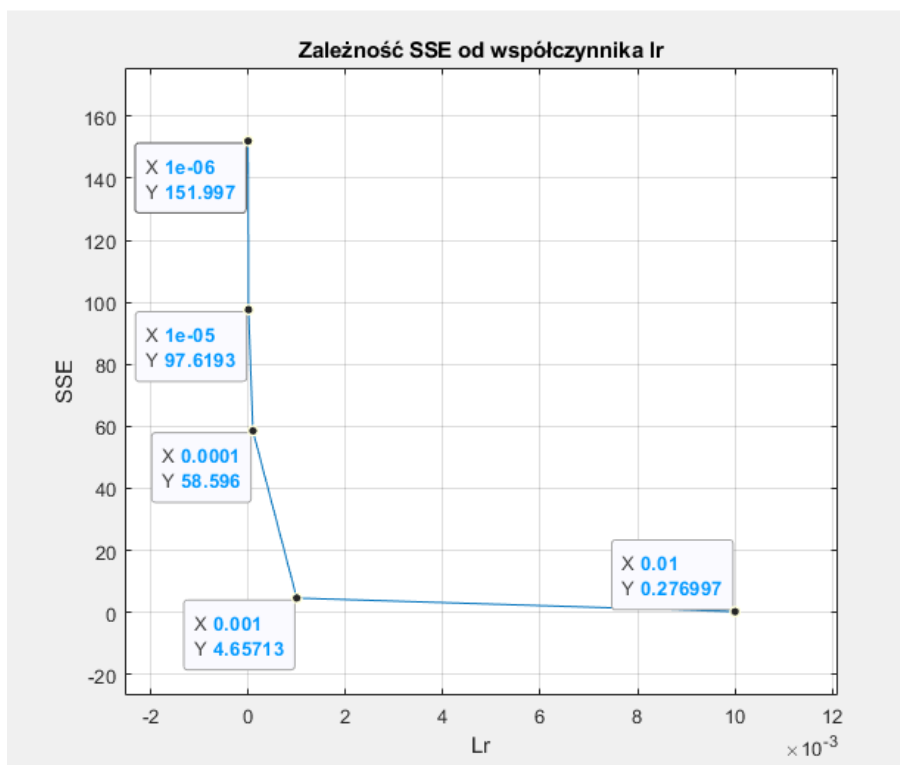
4.9. Eksperyment 9

Eksperyment ten został przeprowadzony dla liczby epokt zmniejszonej do 100 iteracji. Liczba neuronów oraz współczynniki lr pozostały takie same jak w poprzednich eksperymentach.



Rysunek 4.28: Wykres poprawności klasyfikacji dla eksperymentu 9.

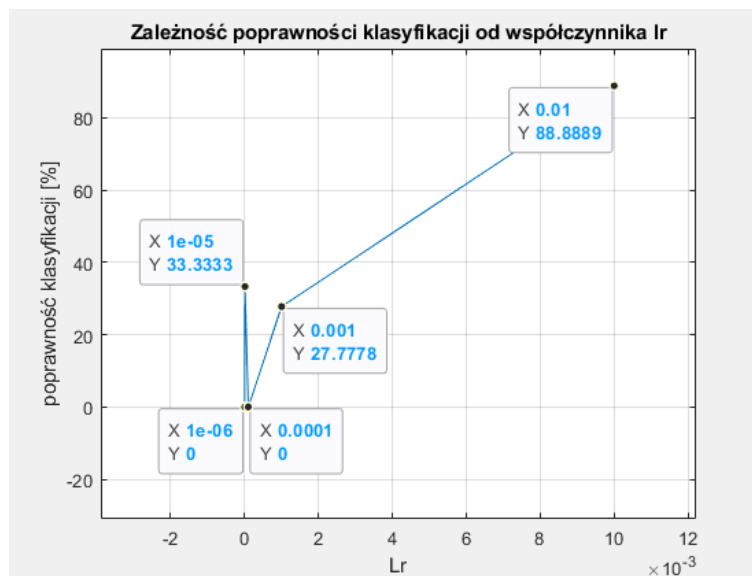
Otrzymane wyniki pozwalają jednoznacznie zauważyć wpływ współczynnika lr na poziom nauczania sieci neuronowej realizowanej na potrzeby tej pracy projektowej. Zauważalny jest również moment niestabilności sieci dla współczynnika $lr = 1e-5$.



Rysunek 4.29: Wykres SSE dla eksperymentu 9.

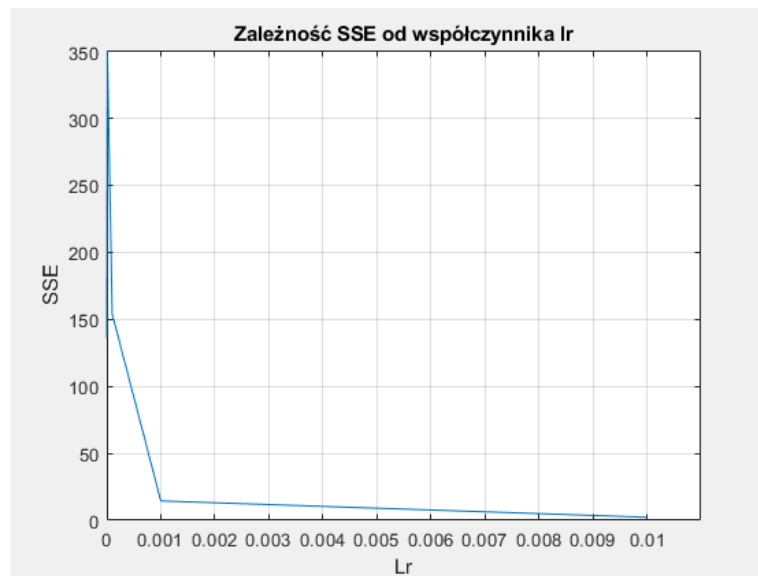
4.10. Eksperyment 10

Ten eksperyment będzie ostatnim z szeregu badań nad działaniem sieci neuronowych. Przeprowadzony został dla ekstremalnie niskiej ilości iteracji równej 10. W przypadku trudniejszych do nauczenia danych eksperyment ten prawdopodobnie zakończyłby się dużym niepowodzeniem. Dla danych wykorzystywanych w tym projekcie zdecydowano iż wyniki takiego eksperymentu mogą być bardzo pouczające.



Rysunek 4.30: Wykres poprawności klasyfikacji dla eksperymentu 10.

Eksperyment ten pokazał wyniki zbliżone do poprzedniego. Widzimy dużą zmianę poziomu nauczania sieci neuronowej w zależności od zmiany współczynnika uczenia lr .



Rysunek 4.31: Wykres SSE dla eksperymentu 10.

5. Wnioski

Na podstawie przeprowadzonych eksperymentów można było dostrzec wiele różnych zachowań sieci w zależności od dobranych parametrów. Zauważyłem iż liczba neuronów w poszczególnych warstwach ukrytych ma wpływ na szybkość nauki sieci neuronowej, im więcej jest tych neuronów tym wolniej sieć uczyła się dla równej ilości epok oraz dla stałego LR. Spowodowane było to zwiększoną ilości wykonywanych przed komputer działań.

Kolejnym aspektem wartym podkreślenia jest fakt iż porównywanie wyników uczenia sieci dla różnych współczynników uczenia przy równej liczbie epok jest dużym spłaszczeniem eksperymentów. Chodzi o to że gdy współczynnik lr jest bardzo mały, sieć potrzebuje więcej czasu (iteracji) do przeprowadzenia pełnego procesu uczenia. Zaś przy większych współczynnikach ten czas był mocno ograniczany. Dlatego na podstawie eksperymentów nastawionych na zmianę współczynnika uczenia jedyną stosowną uwagą będzie pokazanie iż dla określonej liczby iteracji współczynnik lr ma bardzo duże znaczenie. Warto zaznaczyć iż do stworzenia zbliżonego do ideału modelu sieci neuronowej potrzebujemy dobrania ilości iteracji do współczynnika uczenia aby sieć mogła w pełni wykorzystać swoje możliwości nie będąc ograniczaną przez żadne z parametrów.

Co więcej, zauważono iż dla małej ilości neuronów w sieci, sieć ta również powinna posiadać odpowiednio dobraną liczbę iteracji. Dla zbyt małej liczby epok przy niewielkiej liczbie neuronów, sieć nie jest w stanie dostatecznie się nauczyć.

Podsumowując, sieci neuronowe stosowane w tej pracy projektowej posiadały cztery elementy (lr, S1, S2 oraz liczbę epok). Każdy z nich wpływał na wyniki nauczania sieci, lecz jako najważniejszy element można uznać współczynnik lr, gdyż to od niego zależały najbardziej wyniki w przypadku gdy pozostałe elementy zostawały mocno ograniczane.

Literatura

- [1] Osowski S., "Ścież neuronowe do przetwarzania informacji", Oficyna Wydawnicza Politechniki Warszawskiej, Warszawa 2006
- [2] Zajdel R., "Ćwiczenie 6 Model Neuronu", Rzeszów, KLiA, PRz.
- [3] Zajdel R., "Ćwiczenie 8 Sieć jednokierunkowa jednowarstwowa", Rzeszów, KLiA, PRz.
- [4] Zajdel R., "Ćwiczenie 9 Sieć jednokierunkowa wielowarstwowa", Rzeszów, KLiA, PRz.